



Programming in Go(lang)

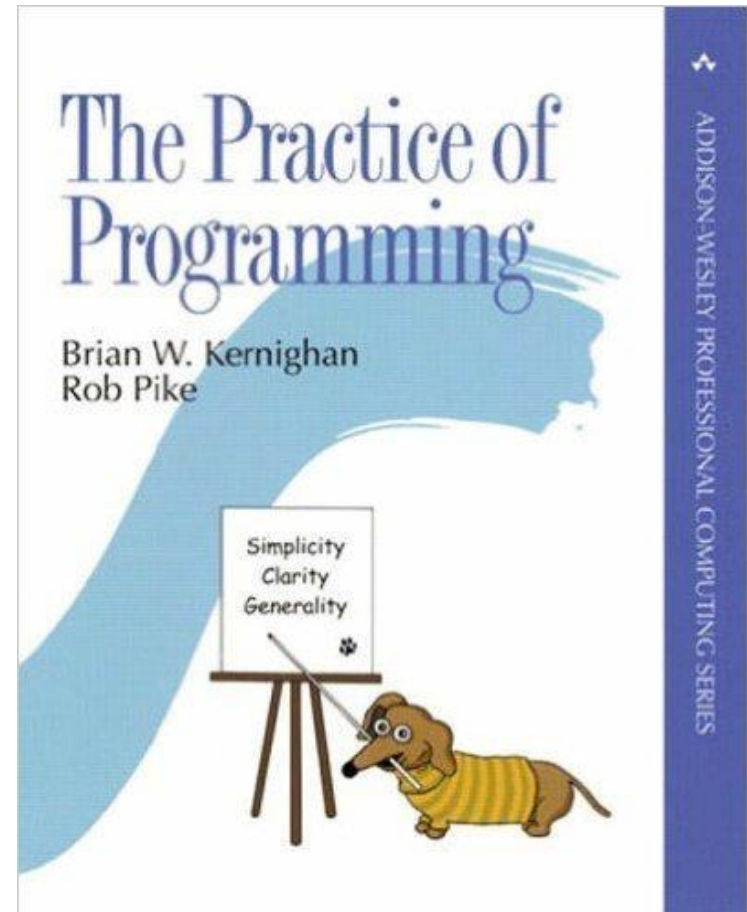
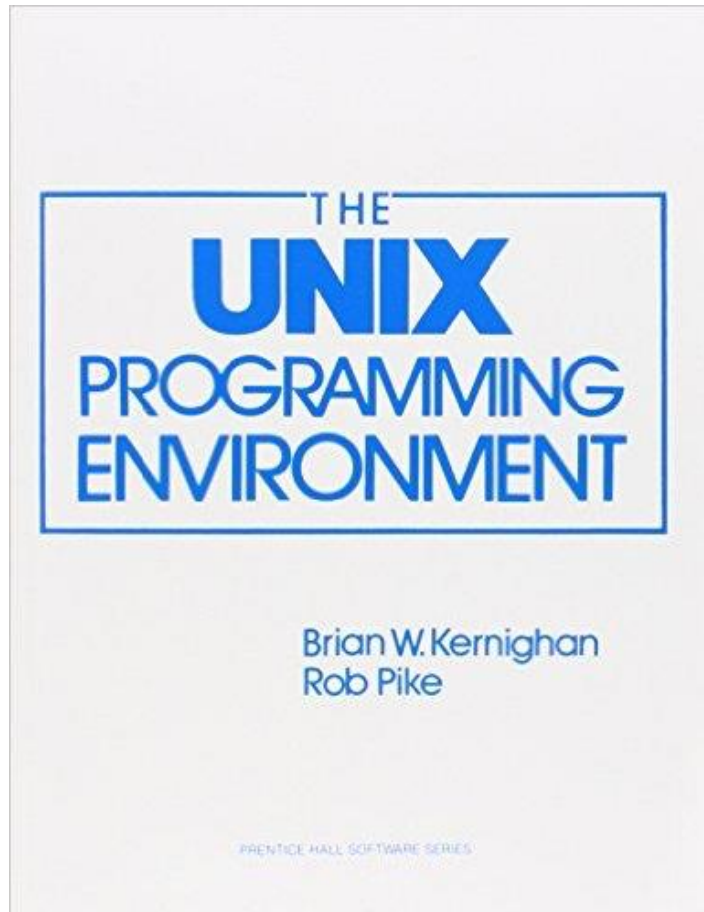
<http://golang.org>



Go timeline

- **Go Language Timeline (English Version)**
- **2007** – Development begins internally at Google.
- **2009** – Public announcement as an open-source project.
- **2012** – Release of Go 1.0, with guaranteed backward compatibility.
- **2015–2019** – Go becomes one of the most popular languages for cloud-native development and microservices.
- **2022** – Introduction of **generics** (Go 1.18).
- **Today** – One of the preferred languages for:
 - high-performance backends
 - distributed systems
 - DevOps tooling
 - container technologies (Docker and Kubernetes are written in Go)

Rob Pike wrote books like...



Ken Thompson wrote operating systems like...





Go

- Designed to be a better version of C++ for the 21st century
- Highly opinionated for efficiency and lower build times
- Became open source in November 2009.
Developed entirely in the open; very active community.

"Go's purpose is therefore not to do research into programming language design; it is to improve the working environment for its designers and their coworkers. Go is more about software engineering than programming language research. Or to rephrase, it is about language design in the service of software engineering."

Rob Pike

What is go?

Simple syntax

Easy to write
concurrent
programs

Compiled
language

Fast compilation -
efficiency

Static linking

Go Tooling

Garbage
collection

Opensource

“Go is an open source programming language that makes it easy to build simple, reliable, and efficient software.”

Go and distributed systems



Go allows you to concentrate on distributed systems problems



good support for concurrency



good support for RPC



garbage-collected (no use after freeing problems)



type safe



Simple language to learn

Go and CLOUD

A language for cloud native applications

Go Cloud: a library and tools for open cloud development in Go

<https://github.com/google/go-cloud>

- Goal: allow application developers to seamlessly deploy cloud applications on any combination of cloud providers
- E.g., read from blob storage (AWS S3 or Google Cloud Storage)

```
ctx := context.Background()
bucket, err := blob.OpenBucket(ctx, "s3://my-bucket")
if err != nil {
    return err
}
defer bucket.Close()
blobReader, err := bucket.NewReader(ctx, "my-blob", nil)
if err != nil {
    return err
}
```


Who uses Go?

Google!

- developed Kubernetes using Go.

Dropbox

- has migrated its performance critical components from Python to Go.r

Mozilla

Facebook

Companies using Go

Organizations in every industry use Go to power their software and services [View all stories](#)



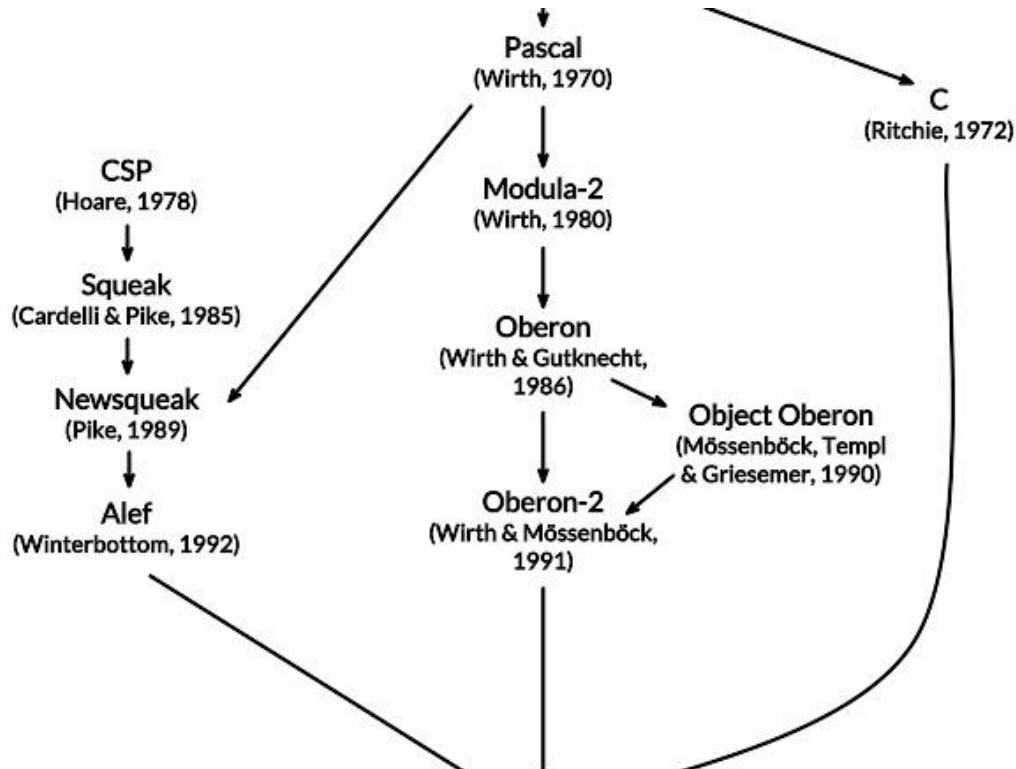
A language for developers

Go was designed by and for people who

- Write large software systems
- Read large software systems
- debug large software systems
- Maintain large software systems.

Go's purpose is

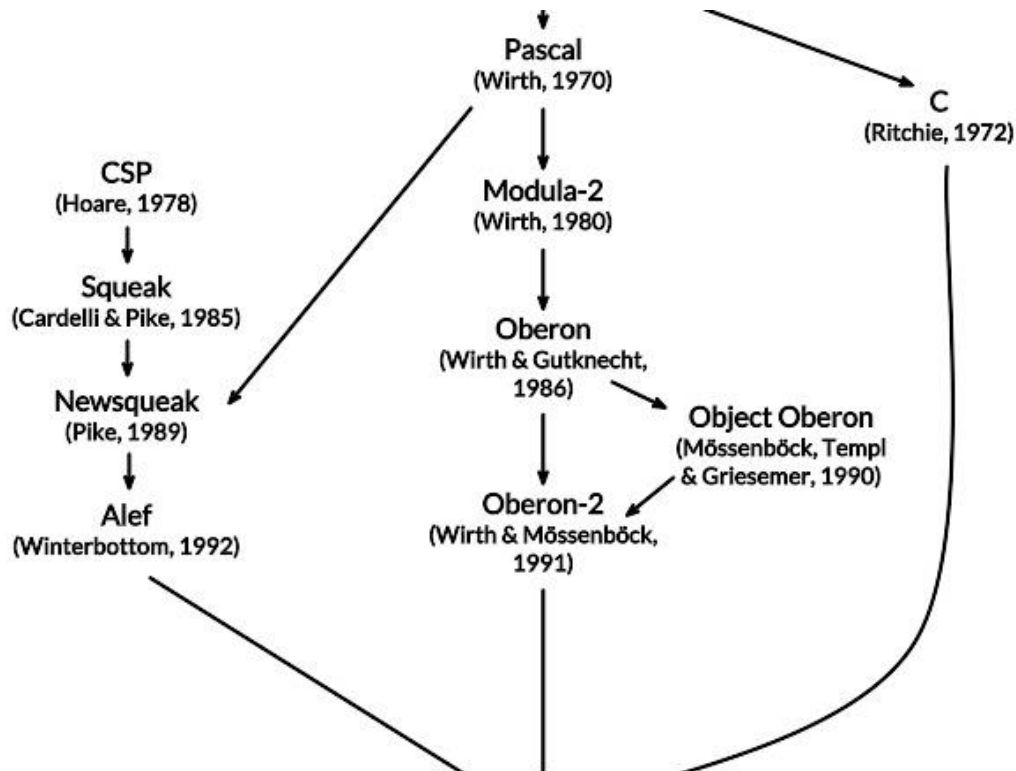
- not research into programming language design
- is to make its designers' programming lives better.



Go: “C-like language” or “C for the 21st century”

Influences

Go’s ancestors: mainly C and CSP (communicating sequential processes) formal language by T. Hoare



Go: "C-like language" or "C for the 21st century"

"When the three of us got started, it was pure research. The three of us got together and decided that we hated C++. We started off with the idea that all three of us had to be talked into every feature in the language, so there was no extraneous garbage put into the language for any reason." (Ken Thompson)

Versions

Stable versions
go1.25.4 (2023-08-08)

The first official stable version, Go 1.0, was released on March 28, 2012, marking an important milestone for the language's toolset.

The first publicly available Go compiler (associated with the Go language) dates back to November 10, 2009, when the project was announced and made open source.

Go is a programming language designed by Google to help solve Google's problems.

Google has big problems.

Eliminate	eliminate slowness
Eliminate	eliminate clumsiness
Improve	improve effectiveness
Maintain	maintain (even improve) scale

GO compiler & ide

IDEs & Editors

- **GoLand** – A professional IDE by JetBrains with smart code completion, refactoring, debugging, and integrated testing.
- **Visual Studio Code + Go Extension** – A free, customizable editor with Go support for modules, debugging, and linting.
- **LiteIDE** – An open-source, lightweight IDE dedicated to Go.

Command-Line Tools

- **go tool (Go 1.24)** – Unified command for managing tools directly from go.mod, with improved caching.
- **Go Modules (go mod)** – Built-in dependency management for Go projects.

Formatting & Style

- **gofmt** – Official Go code formatter.
- **goimports** – Extends gofmt by automatically managing imports.
- **gofumpt** – A stricter variant of gofmt.

Linting & Static Analysis

- **go vet** – Official tool for detecting common issues.
- **staticcheck** – Advanced linter for errors, style, and performance.
- **golangci-lint** – Aggregates multiple linters (govet, errcheck, revive, etc.).
- **awesome-go-linters** – A curated list of over 60 linters and related tools.

Debugging & Testing

- **Delve** – Go debugger supporting breakpoints, variable inspection, and code navigation.
- **go test** – Official testing framework with coverage and benchmarking.

CI/CD & Release

- **Goreleaser** – Automates binary releases, versioning, and distribution.

Package main

```
package main
```

```
import (  
    "fmt"  
    "math/rand"  
)
```

```
func main() {  
    fmt.Println("number is", rand.Intn(10))  
}
```

This program is using the packages with import paths "fmt" and "math/rand".



Every Go program is made up of packages.



Programs start running in package main.



By convention, the package name is the same as the last element of the import path

Exported name

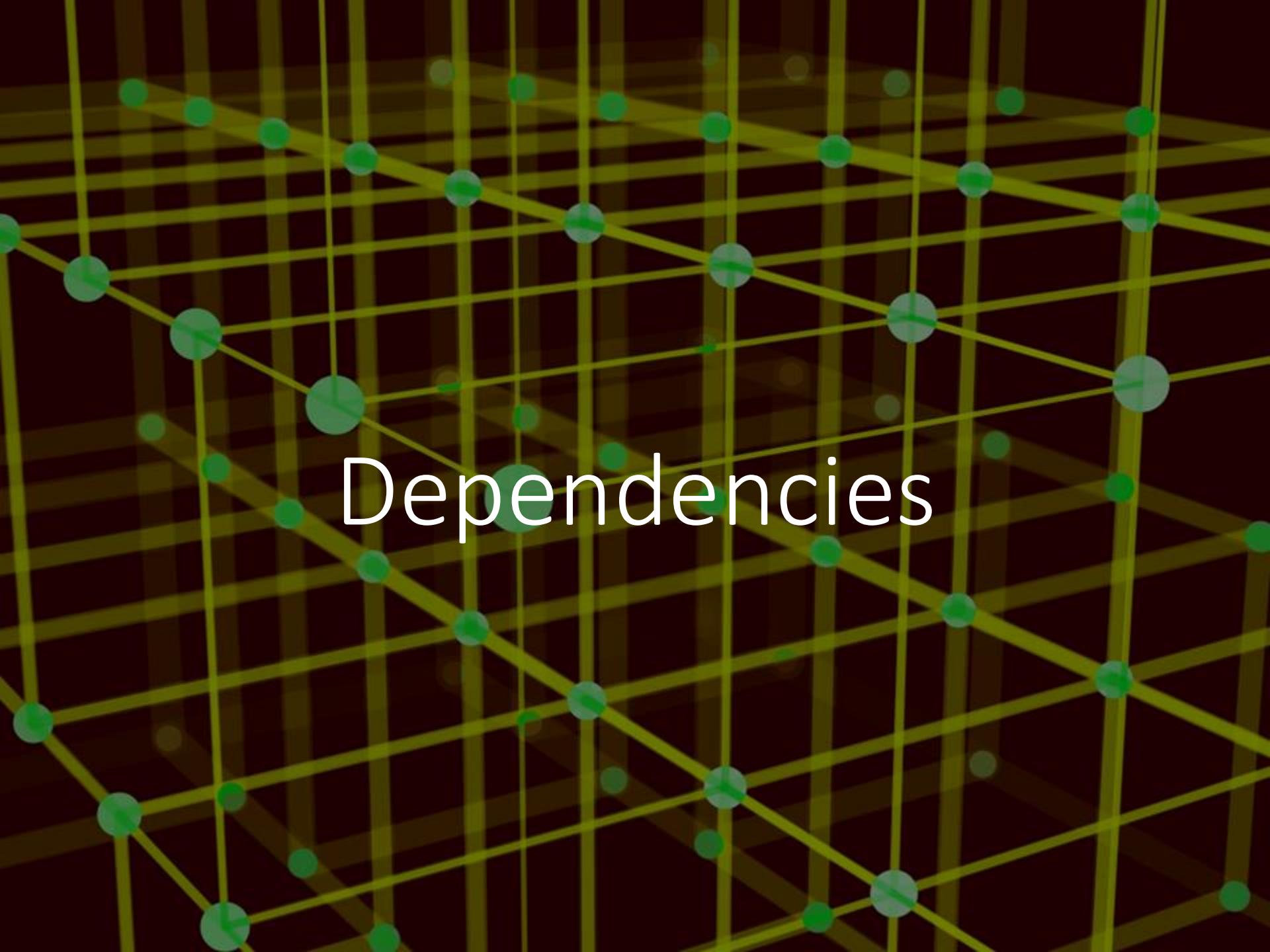
```
package main
```

```
import (  
    "fmt"  
    "math"  
)
```

```
func main() {  
    fmt.Println(math.pi)  
}
```

- In Go, a name is exported if it begins with a capital letter.
- When importing a package, you can refer only to its exported names.
- Any "unexported" names are not accessible from outside the package.

Error: cannot refer to
unexported name math.pi

The background of the slide is a dark brown color. It is overlaid with a complex network of thin, yellow-green lines that intersect to form a grid-like pattern of triangles and quadrilaterals. Scattered throughout this network are numerous small, semi-transparent green circles of varying sizes. Some circles are positioned at the intersections of the lines, while others are in the spaces between them. The overall effect is a sense of interconnectedness and depth.

Dependencies

Dependencies

Dependencies are defined (syntactically) in the language.

Explicit, clear, computable.

```
import "encoding/json"
```

Unused dependencies cause error at compile time.

Efficient: dependencies traversed once per source file...

Circular imports are illegal in Go.

Package name definition

Packages are used to provide code compartmentalisation and reusability.

Package importing

Function

Function name main

func main() – The main is a special function. The program execution starts from the main function. **The main function should always reside in the main package**

GO package

In **Go**, a *package* is a logical unit of code that groups related functions, types, and variables. It helps organize code in a modular and reusable way.

```
package package_name
```

```
// file: util/math.go
package util

func Sum(a int, b int) int {
    return a + b
}
```

```
package main

import (
    "fmt"
    "my_project/util"
)

func main() {
    fmt.Println(util.Sum(3, 4))
}
```

Source file structure

Every source file contains:

- a package clause (which package it's in); that name is the default name used by importers.

`package fmt`

- an optional set of import declarations

`import "fmt" // use default name`

`import myFmt "fmt" // use the name myFmt`

- zero or more global or "package-level" declarations.

Hosting dependencies

Import no transitive

A imports B imports C but A does not directly import C.

The object code for B includes all the information about C needed to import B.

Therefore in A the line import "B" does not require the compiler to read C when compiling A.

The object files are designed so the "export" information comes first;

compiler doing import does not need to read whole file.

Exponentially less data read than with #include files.

With Go in Google, about 40X fanout (recall C++ was 2000x)

Plus in C++ it's general code that must be parsed; in Go it's just export data.

API design

- Through the design of the standard library, great effort spent on controlling dependencies.
- It can be better to copy a little code than to pull in a big library for one function.
- (A test in the system build complains if new core dependencies arise.)
- Dependency hygiene trumps code reuse.

Example:

- The (low-level) net package has own itoa to avoid dependency on the big formatted I/O package.

First example

Each Go program contains a package called main and its main function, after initialization, is where execution starts, analogous with the global main() in C, C++.

The main.main function takes no arguments and returns no value. The program exits - immediately and successfully - when main.main returns.

```
package main // this file is part of package "main»  
import "fmt" // this file uses package "fmt»  
const hello = "Hello, world\n"  
  
func main() {  
    fmt.Print(hello) // fmt is imported pkg's name  
}
```

```
$ go run hello.go    # to compile and run  
$ go build hello.go  # to create a binary
```

Global and package scope

Within a package

- all global variables, functions, types, and constants are visible from all the package's source files.

For clients (importers) of the package,

- names must be upper case to be visible: global variables, functions, types, constants, plus methods and structure fields for global variables and types.

```
const hello = "you smell"           // package visible
```

```
const Hello = "you smell nice"    // globally visible
```

Very different from C/C++: no extern, static, private, public.

Package and program construction: compilation order

Package dependencies determine the order in which to build packages.

Within a package, the source files must all be compiled together.

The package is compiled as a unit, and conventionally each directory contains one package.

Package paths are unique, not package names

The path is "encoding/json" but the package name is json.

```
import "log"           // Standard package
import googlelog "google/base/go/log" //Google-specific package
```

Import package

Project or company name at root of name space.

- `import "google/base/go/log"`

Package name might not be unique; can be overridden.

These are both package log:

Every company might have its own log package; no need to make the package name unique.

Another: there are many server packages in Google's code base.

```
import (  
    "fmt"  
    "log"  
)
```

Remote packages

- Package path syntax works with remote repositories.
- The import path is just a string. Can be a file, can be a URL:

```
go get github.com/4ad/doozer
```

```
// Command to fetch package
```

```
import "github.com/4ad/doozer" // Doozer client's
```

```
import var client doozer.Conn
```

```
// Client's use of package
```

Module

- In Go, a **module** is the **fundamental unit for dependency management**, introduced with the *Go Modules* system (Go 1.11+).
It's broader than a single package: a module is a **collection of related packages**, versioned and identified by a path (usually a repository URL).
- A Module
 - has a **go.mod** file in the root directory.
- Defines:
 - **module name** (e.g., `github.com/user/project`)
 - **Go version** used
 - **dependencies** with their versions.
- Can contain **one or more packages**.

Crafting a module

```
module github.com/vincenza/mio-progetto  
go 1.22  
require (  
    github.com/gin-gonic/gin v1.9.1  
    github.com/pkg/errors v0.9.1  
)
```

```
import "github.com/vincenza/mio-progetto"
```



```
package main
```

```
import (  
    "fmt"  
    "github.com/gin-gonic/gin"  
    "github.com/pkg/errors"  
)
```

```
func main() {  
    r := gin.Default()  
    r.GET("/", func(c *gin.Context) {  
        err := errors.New("example error")  
        c.String(200, fmt.Sprintf("Hello! Error: %v", err))  
    })  
    r.Run(":8080")  
}
```

Directive	Description	Example
<code>module</code>	Defines the module name, usually the repository path.	<code>module github.com/user/project</code>
<code>go</code>	Specifies the Go language version the module is intended to work with.	<code>go 1.22</code>
<code>require</code>	Lists dependencies and their versions that the module needs.	<code>require github.com/gin- gonic/gin v1.9.1</code>
<code>replace</code>	Overrides a dependency with another version or local path (useful for testing or forks).	<code>replace github.com/gin- gonic/gin => ../local-gin</code>
<code>exclude</code>	Excludes a specific version of a dependency to avoid conflicts.	<code>exclude github.com/gin- gonic/gin v1.8.0</code>
<code>retract</code>	Marks a version of the module as deprecated or withdrawn (used by module authors).	<code>retract v1.0.0</code>

Standard Library

- <https://pkg.go.dev/std>



Outline of Go Syntax

- C-like with reversed types and declarations, plus keywords to introduce each type of declaration.
- Syntax is not important... unless you're programming or writing tools
- Tooling is essential, so Go has a clean syntax.
- Not super small, just clean:
 - regular (mostly)
 - only 25 keywords
 - straightforward to parse (no type-specific context required)
- easy to predict, reason about

The background of the image is a complex, three-dimensional molecular structure, possibly representing a protein or a crystalline lattice. It is rendered in shades of blue and white, with a strong sense of depth and perspective. The structure consists of numerous spheres (atoms) connected by rods (bonds). The spheres are highly reflective, showing bright highlights and deep shadows. The rods are also reflective and show some internal detail. The overall effect is a high-tech, scientific aesthetic. The text 'Lexical structure' is superimposed on this background. The word 'Lexical' is in a red, sans-serif font, and the word 'structure' is in a red, serif font. The text is centered horizontally and vertically.

Lexical structure

Comments and Literals

Mostly traditional with modern details.

- Source is UTF-8. White space: blank, tab, newline, carriage return.
- Identifiers are letters and numbers (plus `'_'`) with "letter" and "number" defined by Unicode.

Comments

- `/* This is a comment; no nesting */` or `// So is this`

C-like

- `23`
- `0xFF`
- `1.234e7`

Also `\xNN` always 2 digits;
`\012` always 3; both are bytes:

- `"Hello, world\n"`
- `"\xFF" // 1 byte`
- `"\u00FF" // 1 Unicode char, 2 bytes of UTF-8`

Raw strings

- `\n\abc\t\' == "\\n\\.abc\\t\\"`

Identifiers & KEYWORDS

KEYWORDS

- break default func interface
select case defer go map struct
chan else goto package switch
const fallthrough if range type
continue for import return var

Zero Value

Assigns a default *zero value* to any variable that is declared but not assigned a value.

BLANK IDENTIFIER

- `_`.
- It serves as an anonymous placeholder instead of a regular (non-blank) identifier and has special meaning in declarations, as an operand, and in assignments.

Literals

Integer literals: 1230 1_230 0x23_3

Floating point: 6.03e23. 12_3e3_4

Semicolon

The formal grammar uses semicolons ";" as terminators in a number of productions.

Go programs may omit most of these semicolons using the following two rules:

- When the input is broken into tokens, a semicolon is automatically inserted into the token stream immediately after a line's final token if that token is
 - an identifier
 - an integer, floating-point, imaginary, rune, or string literal
 - one of the keywords `break`, `continue`, `fallthrough`, or `return`
 - one of the operators and punctuation `++`, `--`, `)`, `]`, or `}`

To allow complex statements to occupy a single line, a semicolon may be omitted before a closing `)` or `}`.

Built-in Types

bool: represents a boolean value and is either true or false

Numeric: represents integer types, floating point values, and complex types

string: represents a string value

Derived Types

array: Represents a fixed-size array of elements of the same type.

slice: A dynamically-sized, flexible view into the elements of an array.

map: A collection of key-value pairs.

struct: A composite type that groups together variables (fields) under a single name.

pointer: A variable that stores the memory address of another variable.

interface: A type that specifies a set of method signatures.

Type: predeclared type

Boolean	Numeric Types	String
• <code>bool</code> <pre>a := true b := false</pre>	• <code>int8</code>, <code>int16</code>, <code>int32</code> byte is an alias of <code>uint8</code> rune is an alias of <code>int32</code> • <code>float32</code>, <code>float64</code> • <code>complex64</code>, <code>complex128</code> • <code>Complex</code> • <code>c := 6 + 7i</code> • <code>rune</code>	• <code>string</code> <pre>first := " TEST"</pre>

operators: mostly like C, but `++` and `--` are postfix only and not expressions,
assignment is not an expression

Boolean Data Type

- A boolean data type is declared with the bool keyword and can only take the values true or false.
- Zero value of a boolean data type is false.

Integer Data Types

Type	Size	Range
int	Depends on platform:	-2147483648 to 2147483647 in 32 bit systems and -9223372036854775808 to 9223372036854775807 in 64 bit systems
int8	8 bits/1 byte	-128 to 127
int16	16 bits/2 byte	-32768 to 32767
int32	32 bits/4 byte	-2147483648 to 2147483647
int64	64 bits/8 byte	-9223372036854775808 to 9223372036854775807

Type	Size	Range
uint	Depends on platform	0 to 4294967295 in 32 bit systems and 0 to 18446744073709551615 in 64 bit systems
uint8	8 bits/1 byte	0 to 255
uint16	16 bits/2 byte	0 to 65535
uint32	32 bits/4 byte	0 to 4294967295
uint64	64 bits/8 byte	0 to 18446744073709551615

Float Data Types

Type	Size	Range
float32	32 bits	-3.4e+38 to 3.4e+38.
float64	64 bits	-1.7e+308 to +1.7e+308.

String Data Type

- The string data type is used to store a sequence of characters (text). String values must be surrounded by double quotes

Declarations

Uses Pascal/Modula-style syntax: name before type, more type keywords.

<code>var a int</code>	not	<code>int a;</code>
<code>var fn func([]int)</code>	not	<code>int (*fn)(int[]);</code>
<code>type T struct { a, b int }</code>	not	<code>struct T { int a, b; }</code>

Easier to parse—no symbol table needed.

Tools become easier to write.

One nice effect: can drop `var` and derive type of variable from expression:

```
var buf *bytes.Buffer = bytes.NewBuffer(x) // explicit
buf := bytes.NewBuffer(x)                  // derived
```


Costant

- Constants in Go are
 - are created at compile time, even when defined as locals in functions, and can only be numbers, characters (runes), strings or booleans.
 - Because of the compile-time restriction, the expressions that define them must be constant expressions, evaluatable by the compiler.
 - For instance
 - `1<<3` is a constant expression
 - `math.Sin(math.Pi/4)` is not because the function call to `math`.

Const

Constant declarations are introduced by `const`.

They must have a "constant expression", evaluated at compile time, as initializer and may have an optional type specifier.

```
const Pi = 22./7.  
const AccuratePi float64 = 355./113  
const beef, two, parsnip = "meat", 2, "veg"  
const (  
    Monday, Tuesday, Wednesday = 1, 2, 3  
    Thursday, Friday, Saturday = 4, 5, 6  
)
```

Variable Declarations

- **Declaring a single variable**

```
var name type
```

- ▶ Declaring Multiple variable

```
var name1, name2 type
```

Initialization of local variable

var name type = initialvalue

var name = initialvalue

```
package main
import "fmt"
func main() {
    var age int = 29 // variable declaration with initial value
    fmt.Println("My age is", age)
}
```

```
package main
import "fmt"
func main() {
    var age = 29 // variable declaration with initial value
    fmt.Println("My age is", age)
}
```

Go will automatically be able to infer the type of that variable using that initial value

Initialization of local variable

var name1, name2 type = initialvalue1, initialvalue2

```
package main
import "fmt"
func main() {
    var width, height int = 100, 50
    fmt.Println("width is", width, "heightis", height)
}
```

var name1, name2 = initialvalue1, initialvalue2

```
package main
import "fmt"
func main() {
    var width, height = 100, 50
    fmt.Println("width is", width, "heightis", height)
}
```

Initialization of local variable

```
var (  
    name1 type1 = initialvalue1  
    name2 type2 = initialvalue2  
)
```

```
var name1, name2 = initialvalue1, initialvalue2
```

```
package main
```

```
import "fmt"
```

```
func main() {
```

```
    var width, height = 100, 50
```

```
    fmt.Println("width is", width, "height is", height)
```

```
}
```

Short LOCAL variable declarations

name := initialvalue

name1, name2 := initialvalue1, initialvalue2

```
package main
import "fmt"
func main() {
    var width, height = 100, 50
    fmt.Println("width is", width, "heightis", height)
}
```

:= only inside
function

```
package main
import "fmt"
func main() {
    width, height := 100, 50
    fmt.Println("width is", width, "heightis", height)
}
```

```
field1, offset := nextField(str, 0) field2,
offset := nextField(str, offset) // redeclares offset
a, a := 1, 2 // illegal:
```

Variables can be initialized just like constants but the initializer can be a general expression computed at run time

Zero values

Variables declared without an explicit initial value are given their *zero value*.

The zero value is:

- 0 for numeric types,
- false for the boolean type
- "" (the empty string) for strings.

A close-up, blue-tinted photograph of a financial chart. A silver pen is positioned in the upper right corner, pointing towards the chart. The chart features a jagged line graph with several peaks and troughs. Faint numbers are visible on the chart, including '5' on the left and '2,47' on the right. The word 'Statements' is overlaid in the center in a white, sans-serif font.

Statements

Jump statements

- Return statements
- Break statements - Continue statements
- Fallthrough statements -Goto statements

Simple statements

- Empty statements - Assignments
- Labeled statements - Expression statements
- IncDec statementt - Defer statements

Selection statements

- If statements
- Switch statements
- Select statements

Loop Statements

- For statements

Parallel statements

- Go statements
- Send statements

LIST statements or block

A block is a possibly empty sequence of declarations and statements within matching brace brackets.

In addition to explicit blocks in the source code, there are implicit blocks:

- The universe block encompasses all Go source text.
- Each package has a package block containing all Go source text for that package.
- Each file has a file block containing all Go source text in that file.
- Each "if", "for", and "switch" statement is considered to be in its own implicit block.
- Each clause in a "switch" or "select" statement acts as an implicit block.

Scope & Block

- predeclared identifier scope is the universe block.
- The scope of an identifier denoting a constant, type, variable, or function (but not method) declared at top level (outside any function) is the package block.
- the package name scope of an imported package is the file block of the file containing the import declaration.
- The scope of an identifier declared inside a function begins at the end of the ConstSpec or VarSpec (ShortVarDecl for short variable declarations) and ends at the end of the innermost containing block.
- The scope of a type identifier declared inside a function begins at the identifier in the TypeSpec and ends at the end of the innermost containing block.
- An identifier declared in a block may be redeclared in an inner block. While the identifier of the inner declaration is in scope, it denotes the entity declared by the inner declaration.

Empty & Labeled statements

A labeled statement may be the target of a goto, break or continue statement. Scope of the label is the function where it's declared

```
if x < 5 { }
```

A labeled statement may be the target of a goto, break or continue statement. Scope of the label is the function where it's declared

```
func main() {  
    fmt.Println(1)  
    goto End  
    fmt.Println(2)  
End:  
    fmt.Println(3)  
}
```

Assignments, EXPRESSION & IncDec statements

- With the exception of specific built-in functions, function and method calls and receive operations can appear in statement context.
- Assignment statements may be parenthesized.
- The following built-in functions are not permitted in statement context:
 - `append cap complex imag len make new real`
 - `unsafe.Add unsafe.Alignof unsafe.Offsetof unsafe.Sizeof unsafe.Slice`
- The `"++"` and `"--"` statements increment or decrement their operands by the untyped constant 1.

Assignments, EXPRESSION & IncDec statements

ExpressionList assign_op ExpressionList

Assign_op

- =
- op=.

The assignment proceeds in two phases.

1. the operands of index expressions and pointer indirections
2. the assignments are carried out in left-to-right order.

x, y = 10

x, y = f()

_ = x // evaluate x but ignore it

x, _ = f() // evaluate f() but ignore second result value

a, b = b, a // exchange a and b

Break , Continue & Goto statements

The go, break and continue statements work as in C. They may specify a label to affect an outer structure:

```
func main() {  
    var x int  
    x = 5  
    Loop: for i := 5; i != 0; i-- {  
        fmt.Println(i)  
        switch i {  
            case 4: break Loop  
            case 3: fmt.Println(x)  
        }  
    }  
}
```


FOR

There are three forms:

- For statements with single condition
- For statements with for clause
- For statements with range clause

It unifies for and while and there is no do-while

// Like a C while

for condition { }

// Like a C for

for init; condition; post { }

// Like a C for(;;)

for { }

For with single condition

- simplest form
- "for" statement specifies the repeated execution of a block as long as a boolean condition evaluates to true.
- The condition is evaluated before each iteration. If the condition is absent, it is equivalent to the boolean value true.

```
for a < b { a *= 2 }
```

```
func main() {  
    var x int  
    for {  
        fmt.Println(x)  
        x++  
    }  
}
```

Ciclo infinito

For with **for** clause

- A "for" statement with a ForClause is also controlled by its condition
- additionally it may specify an init and a post statement, such as an assignment, an increment or decrement statement.
- The init statement may be a short variable declaration but the post statement must not. Variables declared by the init statement are re-used in each iteration.

```
for i := 0; i < 10; i++ {  
    f(i) }
```

for cond { S() } is the same as for ; cond ; { S() }
for { S() } is the same as for true { S() }

For with range clause

A "for" statement with a "range" clause iterates through all entries of an array, slice, string or map, or values received on a channel. For each entry it assigns *iteration values* to corresponding *iteration variables* if present and then executes the block.

```
for index, value := range mydatastructure {  
    // Eseguì il codice qui  
}
```

```
numbers := []int{1, 2, 3, 4, 5}
```

```
for index, number := range numbers {  
    fmt.Println(index, number)  
}
```

If

Basic form is familiar, but no dangling else problem:

```
if x < 5 { less() }
```

```
if x < 5 { less() } else if x == 5 { equal() }
```

Initialization statement allowed; requires semicolon.

```
if v := f(); v < 10 {
```

```
    fmt.Printf("%d less than 10\n", v)
```

```
} else {
```

```
    fmt.Printf("%d not less than 10\n", v)
```

```
}
```

Useful with multivariate functions:

```
if n, err = fd.Write(buf); err != nil { ... }
```

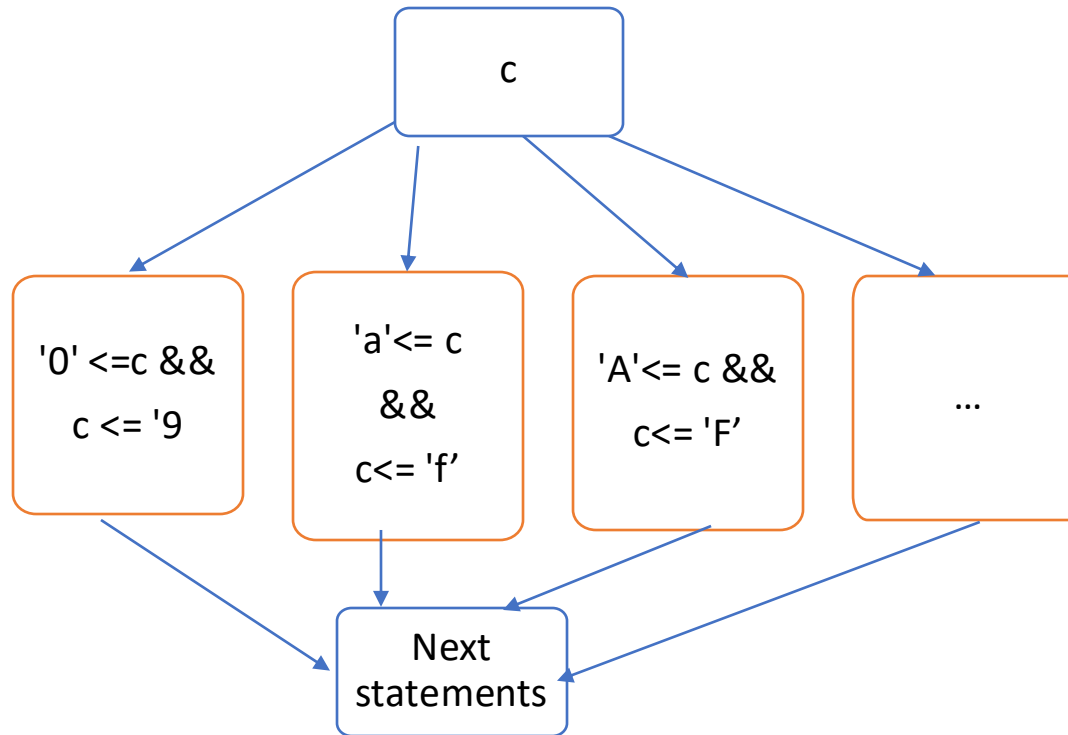
Missing condition means true, which is not too useful in this context but handy in for, switch.

Switch statements

- "Switch" statements provide multi-way execution.
- An expression or type is compared to the "cases" inside the "switch" to determine which branch to execute.
- There are two forms:
 - expression switches: the cases contain expressions that are compared against the value of the switch expression
 - type switches. :the cases contain types that are compared against the type of a specially annotated switch expression.

Expression switches & Fallthrough statements

- A switch statement is a shorter way to write a sequence of if - else statements. It runs the first case whose value is equal to the condition expression.
- Go's switch is like the one in C, C++, Java, JavaScript, and PHP, except that Go only runs the selected case, not all the cases that follow.
- In effect, the break statement that is needed at the end of each case in those languages is provided automatically in Go.
- Another important difference is that Go's switch cases need not be constants, and the values involved need not be integers.



- case expressions need not be constants, are evaluated left-to-right and top-to-bottom;
- the first one that equals the switch expression triggers execution of the statements of the associated case;
- the other cases are skipped.
- If no case matches and there is a "default" case, its statements are executed.
- There can be at most one default case and it may appear anywhere in the "switch" statement.
- A missing switch expression is equivalent to the boolean value true.

- The switch expression type must be comparable.
- If a case expression is untyped, it is first implicitly converted to the type of the switch expression.
- In other words, the switch expression is treated as if it were used to declare and initialize a temporary variable `t` without explicit type; it is that value of `t` against which each case expression `x` is tested for equality.
- In a case or default clause, the last non-empty statement may be a (possibly labeled) "fallthrough" statement to indicate that control should flow from the end of this clause to the first statement of the next clause. Otherwise control flows to the end of the "switch" statement.
- A "fallthrough" statement may appear as the last statement of all but the last clause of an expression switch

```
switch tag {  
    default: s3()  
    case 0, 1, 2, 3: s1()  
    case 4, 5, 6, 7: s2()  
}
```

```
switch x := f(); {  
    // missing switch expression means "true"  
    case x < 0: return -x  
    default: return x  
}
```

```
switch {  
    case x < y: f1()  
    case x < z: f2()  
    case x == 4: f3()  
}
```

Type switch

```
var t interface{}  
t = functionOfSomeType()  
switch t := t.(type) {  
default:  
    fmt.Printf("unexpected type %T\n", t) // %T prints whatever type t has  
case bool:  
    fmt.Printf("boolean %t\n", t) // t has type bool  
case int:  
    fmt.Printf("integer %d\n", t) // t has type int  
case *bool:  
    fmt.Printf("pointer to boolean %t\n", *t) // t has type *bool  
case *int: fmt.Printf("pointer to integer %d\n", *t) // t has type *int }
```

Jump statements

- Return statements
- Break statements - Continue statements
- Fallthrough statements -Goto statements

Simple statements

- Empty statements - Assignments
- Labeled statements - Expression statements
- IncDec statementt - Defer statements

Selection statements

- If statements
- Switch statements
- Select statements

Loop Statements

- For statements

Parallel statements

- Go statements
- Send statements

Type & CAST

Assignability

- x's type is identical to T.
- x's type V and T have identical underlying types and at least one of V or T is not a defined type.
- T is an interface type and x implements T.
- x is a bidirectional channel value, T is a channel type, x's type V and T have identical element types, and at least one of V or T is not a defined type.
- x is the predeclared identifier nil and T is a pointer, function, slice, map, channel, or interface type.
- x is an untyped constant representable by a value of type T.

Type Conversion

Go is very strict about explicit typing.

There is no automatic type promotion or conversion

```
i := 55 //int
```

```
j := 67.8 //float64
```

```
sum := i + j //int + float64 not allowed
```

```
sum := i + int(j) //int + int allowed
```


Numeric conversions

Converting a numeric value from one type to another is a conversion, with syntax like a function call:

```
uint8(intVar) // truncate to size
```

```
int(float64Var) // truncate fraction
```

```
float64(intVar) // convert to float64
```

Also some conversions to and from string:

```
String(0x1234) // == "\u1234"
```

```
string(sliceOfBytes) // bytes -> bytes
```

```
string(sliceOfInts) // ints -> Unicode/UTF-8
```

```
[]byte("abc") // bytes -> bytes
```

Go vs. C expressions

Surprises for the C programmer:

- fewer precedence levels (should be easy)
- ^ instead of ~ (it's binary "exclusive or" made unary)
- ++ and -- are not expression operators

x++ is a statement, not an expression;

- *p++ is (*p) ++ not *(p++)
- &^ is new; handy in constant expressions
- << >> require an unsigned shift count

Non-surprises:

- assignment ops work as expected: += <=& ^= etc.
- expressions generally look the same (indexing, function call, etc.)

Type declaration

Alias declarations

- identifier "=" Type

Examples

type mioltnero int

type (

nodeList = []*Node // nodeList and []*Node are identical types

mioltnero int

- **Type definitions**

Type declaration

Type declarations are introduced by type.

We'll learn more about types later but here are some examples:

```
type Point struct {  
    x, y, z float64  
    name string  
}
```

```
type SliceOfIntPtrs []*int
```

.